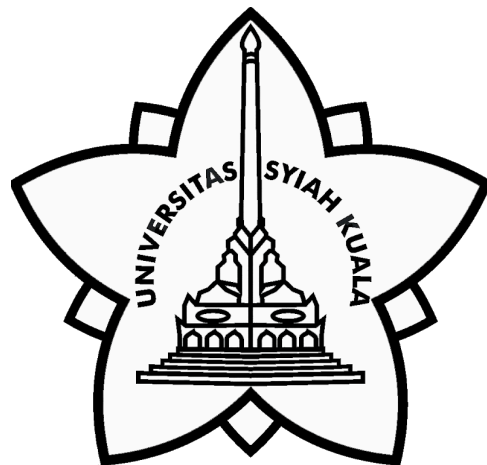


Water Potability Classification

disusun untuk memenuhi tugas
mata kuliah Machine Learning A

Oleh :

Fazhira Rizky Harmayani	2208107010012
Cut Dahliana	2208107010027
Naufal Aqil	2208107010043
Hidayat Nur Hakim	2208107010063
Riska Haqika Situmorang	2208107010086



JURUSAN INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
2025

A. Pemahaman Dataset

Dataset `water_potability.csv` adalah kumpulan data yang sangat berguna untuk analisis klasifikasi dalam bidang kesehatan lingkungan dan pengolahan air. Dataset ini berasal dari situs *Kaggle* dan terdiri dari 3.276 sampel air yang diambil dari berbagai sumber, lengkap dengan 9 fitur numerik yang digunakan untuk menentukan apakah air tersebut layak minum atau tidak layak minum. Label target pada dataset ini adalah kolom `Potability`, yang memiliki dua nilai: 0 (layak) dan 1 (tidak layak). Sumber Dataset. Dataset ini dapat diakses melalui tautan berikut : [Water Quality](#).

Variabel yang Digunakan:

- `ph` : Ukuran keasaman atau kebasaan air. Idealnya 6.5 - 8.5.
- `Hardness` : Jumlah kalsium dan magnesium terlarut.
- `Solids` : Total zat padat terlarut (TDS).
- `Chloramines` : Bahan desinfektan dari kombinasi klorin dan amonia.
- `Sulfate` : Senyawa alami dari tanah dan bebatuan.
- `Conductivity` : Konduktivitas listrik air, dipengaruhi oleh ion terlarut.
- `Organic_carbon` : Kandungan karbon organik dalam air.
- `Trihalomethanes` : Senyawa kimia hasil desinfeksi air.
- `Turbidity` : Ukuran kejernihan air (partikel tersuspensi).
- `Potability` : Target: 1 jika layak minum, 0 jika tidak.

Statistik deskriptif dan visualisasi awal dilakukan untuk memahami distribusi data.

	count	mean	std	min	25%	50%	75%	max
ph	2785.000000	7.080795	1.594320	0.000000	6.093092	7.036752	8.062066	14.000000
Hardness	3276.000000	196.369496	32.879761	47.432000	176.850538	196.967627	216.667456	323.124000
Solids	3276.000000	22014.092526	8768.570828	320.942611	15666.690297	20927.833607	27332.762127	61227.196008
Chloramines	3276.000000	7.122277	1.583085	0.352000	6.127421	7.130299	8.114887	13.127000
Sulfate	2495.000000	333.775777	41.416840	129.000000	307.699498	333.073546	359.950170	481.030642
Conductivity	3276.000000	426.205111	80.824064	181.483754	365.734414	421.884968	481.792304	753.342620
Organic_carbon	3276.000000	14.284970	3.308162	2.200000	12.065801	14.218338	16.557652	28.300000
Trihalomethanes	3114.000000	66.396293	16.175008	0.738000	55.844536	66.622485	77.337473	124.000000
Turbidity	3276.000000	3.966786	0.780382	1.450000	3.439711	3.955028	4.500320	6.739000
Potability	3276.000000	0.390110	0.487849	0.000000	0.000000	0.000000	1.000000	1.000000

Gambar 1. Statistik deskriptif dan visualisasi awal dilakukan untuk memahami distribusi data.

Statistik ini menunjukkan bahwa sebagian besar fitur memiliki variasi nilai yang cukup luas, dengan beberapa kemungkinan outlier dan data hilang. Rata-rata pH netral (7.08), kekerasan air 196.37, dan kandungan zat padat terlarut (solids) cukup tinggi di 22.014 ppm. Chloramines dan sulfate masing-masing memiliki rata-rata 7.12 mg/L dan 333.78 mg/L, dengan distribusi yang cukup lebar. Konduktivitas air rata-rata 426.21 $\mu\text{S}/\text{cm}$, organic carbon 14.28 mg/L, trihalomethanes 66.4 $\mu\text{g}/\text{L}$ (dengan data hilang), dan turbidity relatif stabil di angka 3.97 NTU. Hanya sekitar 39% sampel yang diklasifikasikan sebagai air layak minum, menunjukkan pentingnya proses klasifikasi dan pengolahan lebih lanjut.

B. Eksplorasi Data dan Pra-pemrosesan

Langkah-langkah yang dilakukan dalam eksplorasi data meliputi:

- Mengecek nilai Unik

```
# Menampilkan jumlah nilai unik dalam dataset
print("\nJumlah Nilai Unik per Kolom:")
print(water_df.nunique())
```


Jumlah Nilai Unik per Kolom:	
ph	2785
Hardness	3276
Solids	3276
Chloramines	3276
Sulfate	2495
Conductivity	3276
Organic_carbon	3276
Trihalomethanes	3114
Turbidity	3276
Potability	2
dtype: int64	

Gambar 2. Kode Mengecek Missing Values

Pada gambar diatas, Mayoritas parameter dalam dataset *Water Potability* memiliki jumlah nilai unik yang mendekati total jumlah sampel (maksimum 3.276), yang menunjukkan tingkat presisi pengukuran yang tinggi pada sebagian besar fitur. Namun, fitur seperti pH dan Sulfate memiliki jumlah nilai unik yang lebih sedikit, hal

ini disebabkan oleh keberadaan data yang hilang (*missing values*). Sementara itu, variabel target Potability bersifat biner (0 atau 1), menjadikannya sangat cocok untuk diterapkan dalam model klasifikasi guna memprediksi kelayakan air untuk dikonsumsi.

- **Mengecek Missing Value**

```
# Mengecek jumlah missing values
print("\nMissing Values dalam Dataset:")
print(water_df.isnull().sum())
```



```
Missing Values dalam Dataset:
ph                491
Hardness           0
Solids             0
Chloramines        0
Sulfate           781
Conductivity       0
Organic_carbon     0
Trihalomethanes   162
Turbidity          0
Potability         0
dtype: int64
```

Gambar 3. Kode mengecek missing value

Dari gambar diatas dapat dilihat Fitur pH memiliki sebanyak 491 nilai yang hilang, sedangkan Sulfate mencatat jumlah tertinggi dengan 781 missing values. Selain itu, fitur Trihalomethanes juga memiliki 162 data yang tidak tersedia. Kehadiran nilai-nilai yang hilang ini dapat memengaruhi performa model jika tidak ditangani dengan tepat, sehingga diperlukan strategi penanganan seperti imputasi atau penghapusan baris tertentu selama proses analisis.

- Mengecek data duplikat

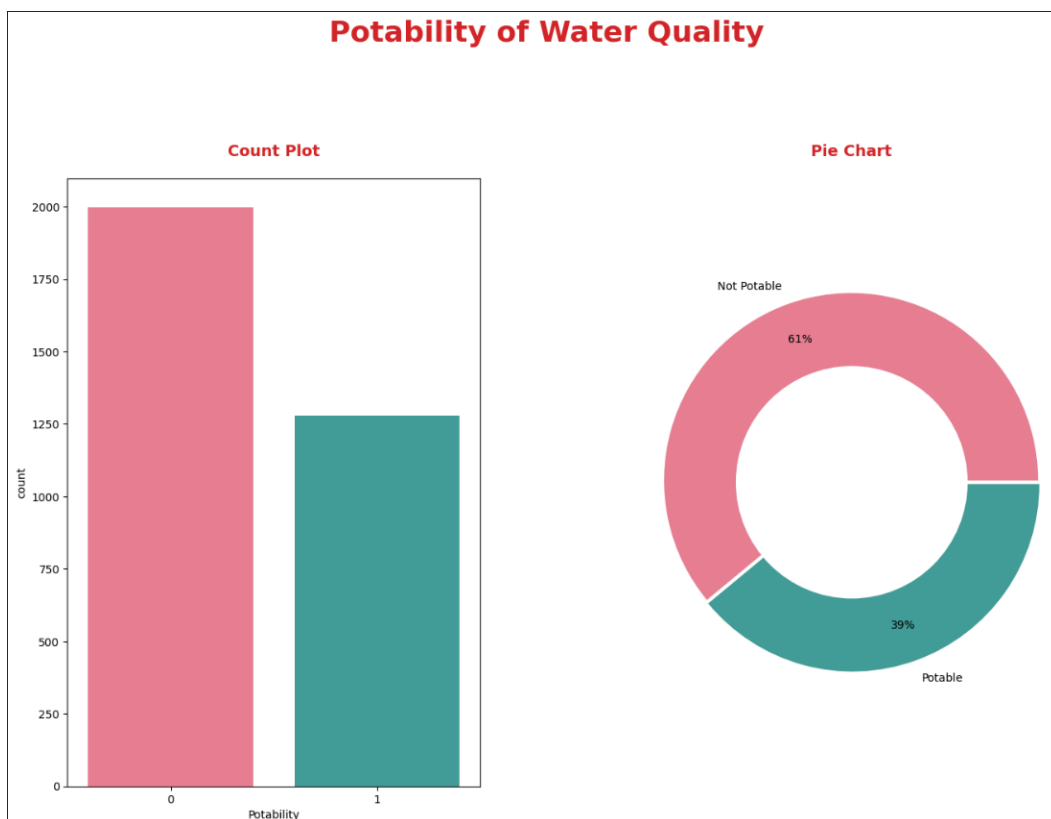
```
# Menampilkan data duplikat
print("\nData Duplikat dalam Dataset:")
print(water_df.duplicated().sum())

Data Duplikat dalam Dataset:
0
```

Gambar 4. Kode Mengecek data duplikat

Dapat dilihat dari gambar diatas bahawa tidak data duplikat

- Visualisasi distribusi target



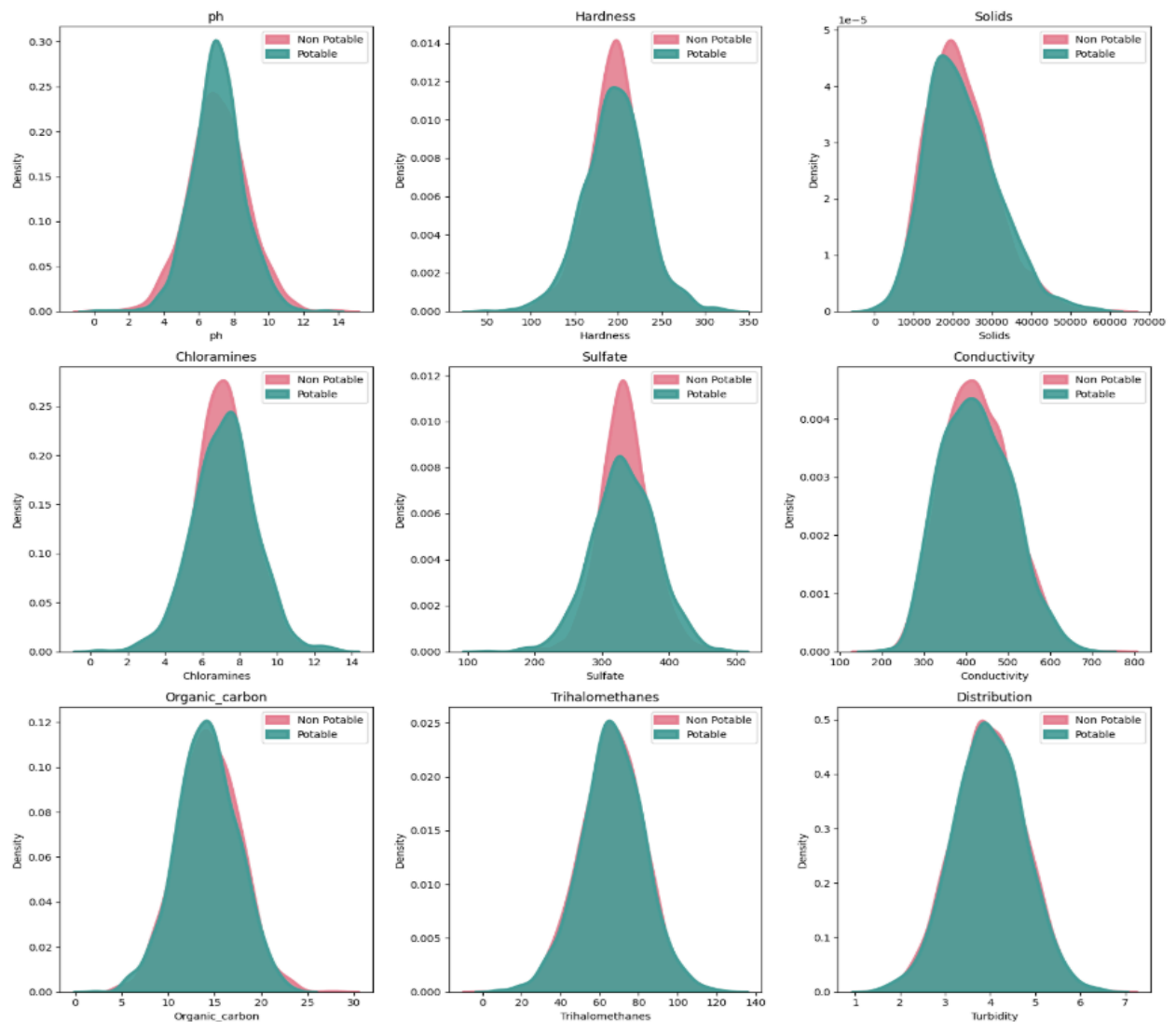
Gambar 5. Visualisasi distribusi kelas target

Dari gambar diatas dapat dilihat bahwa Dataset *Water Potability* menunjukkan adanya ketidakseimbangan kelas yang cukup signifikan pada distribusi variabel target, dengan selisih sekitar 20% antara kelas air layak minum (1) dan tidak layak (0).

Ketimpangan ini merupakan kondisi yang umum ditemukan dalam data kualitas air di dunia nyata, mengingat air dari sumber alami atau yang belum melalui proses pengolahan lebih sering tidak memenuhi standar kelayakan konsumsi. Ketidakseimbangan ini memiliki implikasi penting dalam pembangunan model prediksi, karena dapat memengaruhi performa model, terutama dari sisi akurasi dan keadilan klasifikasi.

- **Visualisasi distribusi fitur**

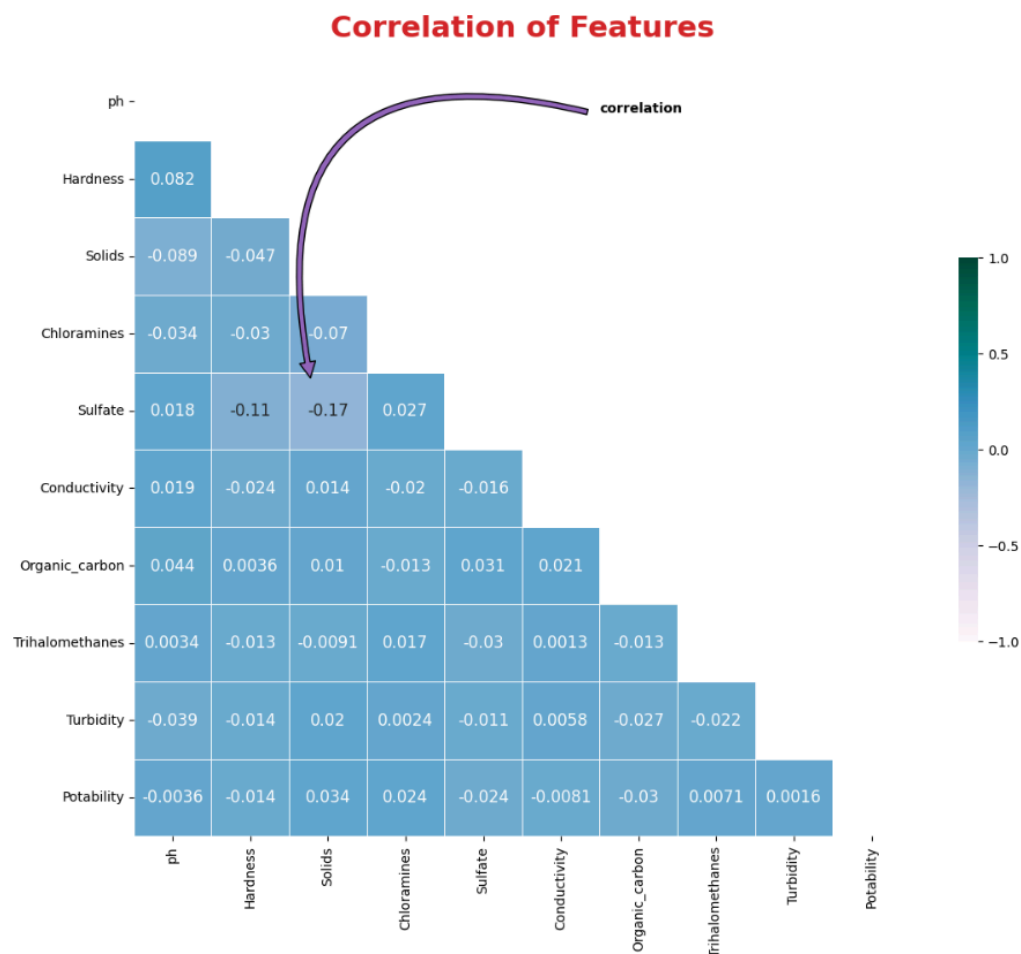
Distribution of features



Gambar 6. *Visualisasi distribusi fitur*

Dari gambar diatas dapat dilihat dalam analisis distribusi data pada dataset *Water Potability*, sebagian besar fitur menunjukkan pola distribusi yang mendekati normal. Namun, beberapa fitur seperti Solids, Organic_carbon, dan Conductivity tampak memiliki skewness ke kanan (right-skewed) yang cukup mencolok, mencerminkan adanya konsentrasi nilai rendah dengan sebagian kecil nilai yang sangat tinggi. Selain itu, seluruh fitur diketahui mengandung outlier, yang tetap dipertahankan dalam analisis karena dianggap sebagai data valid yang merepresentasikan kondisi ekstrim dari kualitas air di dunia nyata. Perlu juga diperhatikan adanya perbedaan skala antar fitur—misalnya, nilai Solids dapat mencapai angka sekitar 20.000, sementara pH hanya berkisar di angka 7. Perbedaan ini berpengaruh besar dalam proses modeling, sehingga penting untuk melakukan normalisasi atau standardisasi agar setiap fitur dapat diperlakukan secara seimbang oleh algoritma machine learning.

- **Visualisasi Korelasi fitur**



Gambar 7. *Visualisasi Korelasi antar fitur*

Dari gambar diatas dapat dilihat Hasil analisis korelasi pada dataset *Water Potability* menunjukkan bahwa sebagian besar fitur memiliki korelasi yang rendah atau tidak signifikan satu sama lain, menandakan bahwa fitur-fitur tersebut bersifat relatif independen. Selain itu, tidak ditemukan adanya hubungan linier yang kuat antara fitur-fitur numerik dengan variabel target Potability. Kondisi ini mengindikasikan bahwa pola hubungan antara karakteristik air dan kelayakannya untuk dikonsumsi bersifat kompleks, sehingga pendekatan berbasis machine learning sangat diperlukan untuk mengidentifikasi pola non-linier yang tidak dapat ditangkap secara langsung melalui analisis statistik sederhana.

Langkah-langkah yang dilakukan dalam pra pemrosesan data meliputi:

- **Menghapus nilai kosong (Missing Value)**

```
water_df = water_df.dropna()
```

Gambar 8. *Kode menghapus nilai kosong*

Kode diatas digunakan untuk menghapus seluruh baris dalam dataset yang memiliki nilai kosong (missing values). Langkah ini dilakukan agar data yang digunakan untuk pelatihan model bersih dan bebas dari nilai yang tidak lengkap,

```
water_df.isnull().sum()

[5]
...  ph          0
     Hardness    0
     Solids      0
     Chloramines  0
     Sulfate     0
     Conductivity 0
     Organic_carbon 0
     Trihalomethanes 0
     Turbidity    0
     Potability   0
     dtype: int64
```

Gambar 9. *Output penanganan missing value*

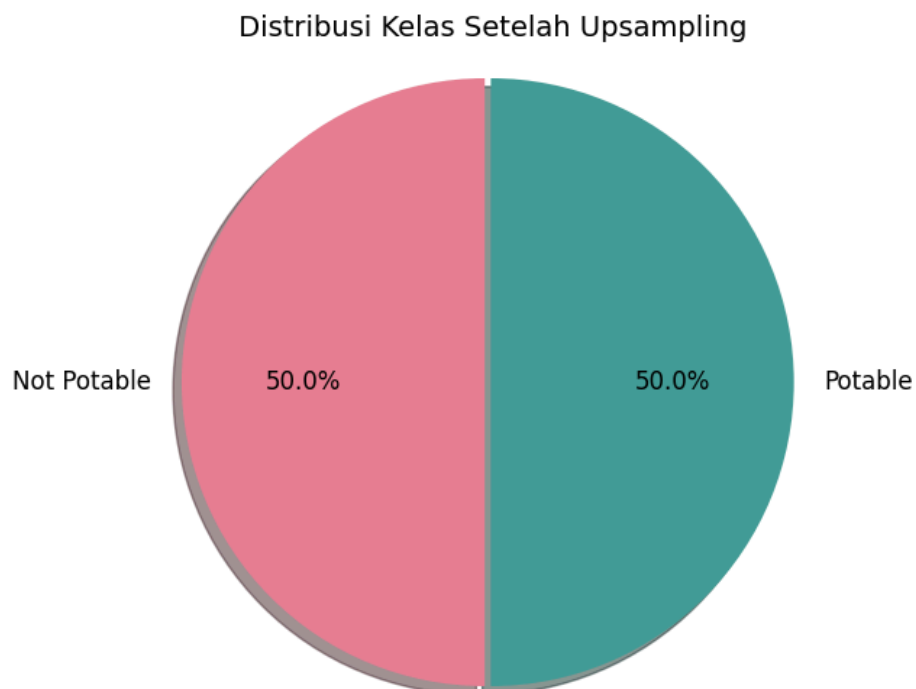
Setelah dilakukan proses pembersihan data, seluruh fitur dalam dataset *Water Potability* kini telah bebas dari nilai kosong (missing values). Hal ini menandakan bahwa dataset sudah dalam kondisi bersih dan siap digunakan untuk tahap pemodelan machine learning tanpa perlu lagi melakukan imputasi atau penghapusan data tambahan.

- **Menyeimbangkan Distribusi Kelas (Upsampling)**

```
# Menyeimbangkan kelas (upsampling class 1)
from sklearn.utils import resample, shuffle
df_0 = water_df[water_df['Potability'] == 0]
df_1 = water_df[water_df['Potability'] == 1]
df_1_upsampled = resample(df_1, replace=True, n_samples=len(df_0), random_state=42)
df_balanced = shuffle(pd.concat([df_0, df_1_upsampled]), random_state=42)
```

Gambar 10. Kode menyeimbangkan distribusi kelas

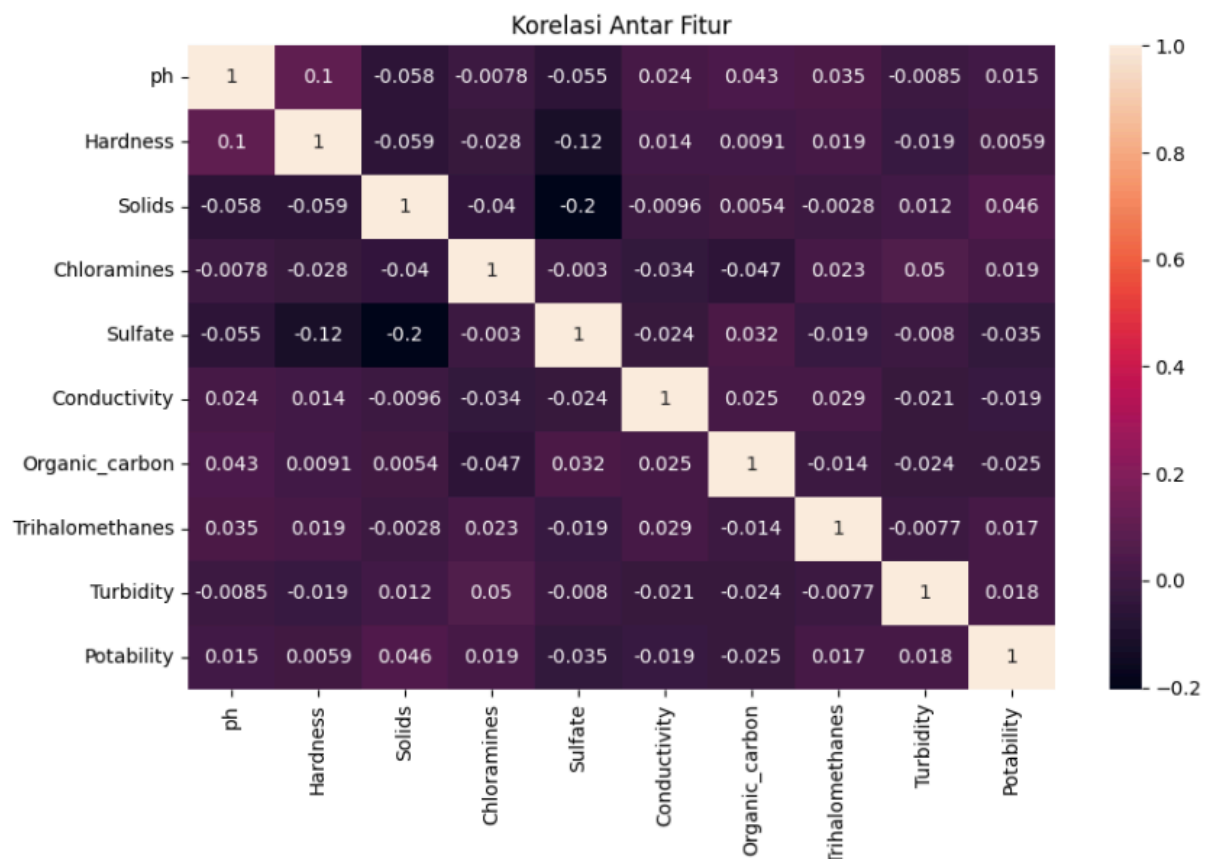
- **Visualisasi Distribusi Kelas (Upsampling)**



Gambar 11. visualisasi distribusi kelas setelah upsampling

Scatter plot ini menunjukkan hubungan antara Harga, Diskon, Rating, Jumlah Ulasan, dan Sales. Hasilnya, tidak tampak korelasi linear yang kuat antar variabel. Distribusi data cukup bervariasi, terutama pada Sales, Price, dan NumReviews. Visualisasi ini membantu memahami bahwa hubungan antar variabel mungkin bersifat kompleks dan perlu dianalisis lebih lanjut dengan metode statistik atau pemodelan. Setelah dilakukan proses penyeimbangan data, distribusi untuk masing-masing kelas pada variabel target Potability kini menjadi 50.0% untuk kelas 0 (tidak layak minum) dan 50.0% untuk kelas 1 (layak minum). Hal ini menunjukkan bahwa strategi balancing yang diterapkan berhasil menciptakan distribusi kelas yang seimbang, sehingga dapat membantu mengurangi bias model terhadap mayoritas kelas dan meningkatkan performa klasifikasi secara keseluruhan, khususnya pada metrik seperti recall dan F1-score.

- **Visualisasi korelasi fitur setelah dibersihkan**



Gambar 12. Visualisasi korelasi antar fitur setelah di bersihkan

Analisis korelasi dalam dataset *Water Potability* menunjukkan bahwa tidak ada satu pun fitur yang secara dominan berkorelasi dengan variabel target Potability, sehingga hubungan antar fitur dan target tidak dapat dijelaskan hanya dengan pendekatan statistik sederhana. Oleh karena itu, penggunaan model machine learning menjadi penting untuk mengungkap pola-pola kompleks dan non-linier dalam data. Salah satu kelebihan dari hasil ini adalah tidaknya ditemukan multikolinearitas yang kuat antar fitur, yang berarti semua fitur dapat tetap digunakan dalam proses pemodelan tanpa perlu penghapusan atau reduksi dimensi, sehingga model memiliki peluang lebih besar untuk menangkap kontribusi unik dari masing-masing fitur.

- **Korelasi Fitur terhadap Target Potability**

```
df_balanced.corr().abs()['Potability'].sort_values(ascending = False)
```

Potability	1.000000
Solids	0.045535
Sulfate	0.035401
Organic_carbon	0.025118
Chloramines	0.018753
Conductivity	0.018581
Turbidity	0.017854
Trihalomethanes	0.017463
ph	0.014620
Hardness	0.005864

Name: Potability, dtype: float64

Gambar 13. korelasi fitur terhadap target

Berdasarkan hasil korelasi terhadap variabel target Potability, fitur-fitur dalam dataset *Water Potability* dapat disusun dari yang memiliki pengaruh paling besar hingga yang paling rendah sebagai berikut: Solids (0.0455), Sulfate (0.0354), Organic_carbon (0.0251), Chloramines (0.0188), Conductivity (0.0186), Turbidity (0.0179), Trihalomethanes (0.0175), ph (0.0146), dan Hardness (0.0059). Meskipun seluruh nilai korelasi tergolong rendah, urutan ini memberikan gambaran awal mengenai fitur-fitur yang relatif lebih berkaitan dengan kelayakan air untuk diminum. Namun, karena tidak ada fitur dengan korelasi yang kuat terhadap target, metode machine learning tetap diperlukan untuk memahami hubungan kompleks yang mungkin tersembunyi antar variabel.

- **Standarisasi fitur**

```
# === SCALING FITUR ===  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Gambar 14. *Kode scaling fitur*

Proses standarisasi fitur dilakukan untuk menyelaraskan skala seluruh variabel numerik agar tidak ada fitur yang mendominasi model hanya karena memiliki nilai absolut yang lebih besar. Hal ini sangat penting, terutama saat menggunakan algoritma seperti K-Nearest Neighbors (KNN) dan Logistic Regression, yang sensitif terhadap perbedaan skala antar fitur. Sebagai contoh, nilai awal pH sebesar 7.0 dapat berubah menjadi sekitar 0.15 setelah melalui proses standarisasi. Dengan demikian, semua fitur akan memiliki distribusi yang seragam, umumnya dengan rata-rata 0 dan standar deviasi 1, sehingga model dapat mengevaluasi kontribusi masing-masing fitur secara lebih adil dan seimbang.

- **Kesimpulan: Eksplorasi Data dan Prapemrosesan**

Melalui proses eksplorasi data terhadap dataset *Water Potability*, diperoleh pemahaman awal bahwa sebagian besar fitur memiliki nilai unik yang tinggi dan distribusi yang mendekati normal, meskipun beberapa fitur menunjukkan skewness serta keberadaan outlier. Selain itu, ditemui pula nilai-nilai kosong (missing values) pada beberapa fitur penting seperti *pH*, *Sulfate*, dan *Trihalomethanes*, yang berpotensi mengganggu performa model jika tidak ditangani. Analisis terhadap variabel target mengungkapkan adanya ketidakseimbangan kelas, yang dapat menyebabkan bias pada model klasifikasi apabila tidak diperbaiki. Hasil korelasi antar fitur juga menunjukkan bahwa tidak terdapat hubungan linier yang kuat, baik antar fitur maupun terhadap target, sehingga pendekatan statistik sederhana tidak memadai dalam memahami pola dalam data ini.

C. Implementasi Model

- Pisahkan fitur dan target

```
# === DEFINISI FITUR DAN TARGET ===  
X = df_balanced.drop('Potability', axis=1)  
y = df_balanced['Potability']
```

Gambar 15. Kode Pisahkan Fitur (x) dan Target (y)

Gambar diatas dipisahkan menjadi dua bagian utama: X yang berisi fitur numerik seperti pH, Hardness, Solids, Sulfate, dan lainnya, serta y yang merupakan variabel target Potability dengan nilai biner, yaitu 0 untuk air tidak layak minum dan 1 untuk air layak minum. Pemisahan ini penting untuk memisahkan input dan output dalam model machine learning.

- Bagi Data menjadi Training dan Testing (80%-20%)

```
# === SPLIT DATA TRAIN-TEST ===  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.1, random_state=42, stratify=y)
```

Gambar 16. Kode Bagi Data menjadi Training dan Testing (80%-20%)

Sebanyak 80% data digunakan untuk proses pelatihan (training), sementara 20% sisanya digunakan untuk pengujian (testing). Pemisahan ini bertujuan untuk mengevaluasi performa model pada data yang belum pernah dilihat sebelumnya, sehingga dapat mengukur kemampuan generalisasi model secara lebih objektif.

- Evaluasi Awal Sebelum Tuning

```
# === INISIALISASI MODEL (SEBELUM TUNING) ===
models = {
    'Logistic Regression': LogisticRegression(),
    'KNN': KNeighborsClassifier(),
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier()
}

print("=== Akurasi Sebelum Tuning ===")
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"{name}: {accuracy_score(y_test, y_pred):.2f}")
```

Gambar 17. Kode Inisialisasi Sebelum Tuning

Sebelum dilakukan proses tuning hyperparameter, langkah awal yang dilakukan adalah menginisialisasi empat model klasifikasi dasar, yaitu Logistic Regression, K-Nearest Neighbors (KNN), Naïve Bayes, dan Decision Tree.

Model	Akurasi Awal
Logistic Regression	51% ✗ (tidak lebih baik dari tebak acak)
KNN	75% ✓ (sudah cukup bagus)
Naive Bayes	53% ✗
Decision Tree	81% ✓ (paling bagus sebelum tuning)

Gambar 18. Hasil Output Kode Inisialisasi Sebelum Tuning

Berdasarkan hasil awal yang ditampilkan pada gambar, diketahui bahwa Decision Tree memberikan akurasi tertinggi sebesar 81%, disusul oleh KNN dengan 75%. Sementara itu, Logistic Regression dan Naïve Bayes hanya menghasilkan akurasi masing-masing 51% dan 53%, yang menunjukkan bahwa keduanya belum mampu menangkap pola dalam data secara optimal. Hasil ini memberikan gambaran awal performa masing-masing model dan

menjadi dasar untuk dilakukan tuning dan evaluasi lanjutan pada tahap berikutnya.

- Tuning Hyperparameter Model

```
# === INISIALISASI UNTUK TUNING ===
results = {}
best_models = {}

# === LOGISTIC REGRESSION TUNING ===
print("\n☛ Sedang melakukan tuning pada model Logistic Regression ...")
lr = LogisticRegression()
param_lr = {'C': [0.01, 0.1, 1, 10], 'solver': ['liblinear', 'lbfgs']}
grid_lr = GridSearchCV(lr, param_lr, cv=5)
grid_lr.fit(X_train, y_train)
best_models['Logistic Regression'] = grid_lr.best_estimator_
results['Logistic Regression'] = grid_lr.best_score_

# === KNN TUNING ===
print("\n☛ Sedang melakukan tuning pada model KNN ...")
knn = KNeighborsClassifier()
param_knn = {'n_neighbors': np.arange(1, 50), 'weights': ['uniform', 'distance']}
grid_knn = GridSearchCV(knn, param_knn, cv=5)
grid_knn.fit(X_train, y_train)
best_models['KNN'] = grid_knn.best_estimator_
results['KNN'] = grid_knn.best_score_

# === NAIVE BAYES (TIDAK PERLU TUNING) ===
print("\n☛ Tidak melakukan tuning Naive Bayes ...")
nb = GaussianNB()
nb.fit(X_train, y_train)
best_models['Naive Bayes'] = nb
results['Naive Bayes'] = nb.score(X_train, y_train)

# === DECISION TREE TUNING ===
print("\n☛ Sedang melakukan tuning pada model Decision Tree ...")
dt = DecisionTreeClassifier()
param_dt = {
    'criterion': ['gini', 'entropy'],
    'max_depth': np.arange(1, 50),
    'min_samples_leaf': [1, 2, 4, 5, 10, 20, 30, 40, 80, 100]
}
grid_dt = GridSearchCV(dt, param_dt, cv=5)
grid_dt.fit(X_train, y_train)
best_models['Decision Tree'] = grid_dt.best_estimator_
results['Decision Tree'] = grid_dt.best_score_
```

Gambar 19. Kode Tuning Hyperparameter Model

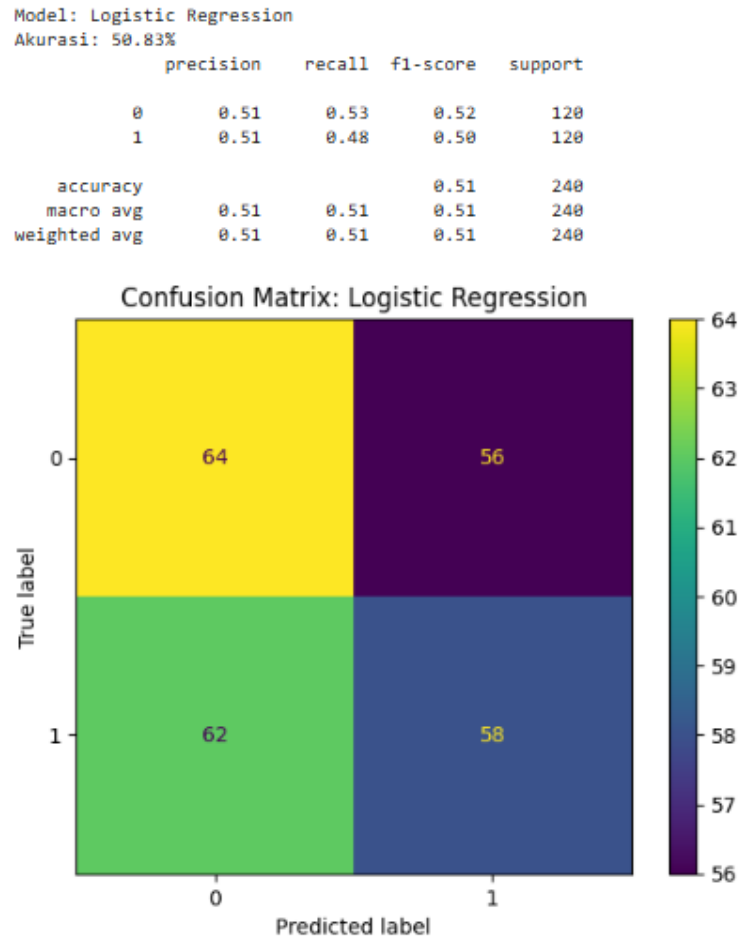
Setelah melakukan evaluasi awal terhadap performa masing-masing model, langkah berikutnya adalah melakukan proses tuning hyperparameter untuk meningkatkan akurasi model. Tuning dilakukan menggunakan teknik Grid Search Cross Validation (GridSearchCV) pada tiga dari empat model, yaitu Logistic Regression, K-Nearest Neighbors (KNN), dan Decision Tree.

Untuk Naïve Bayes, tidak dilakukan tuning karena model ini tidak memiliki banyak parameter yang dapat ditentukan secara manual dan cenderung sensitif terhadap perubahan data daripada terhadap parameter.

D. Evaluasi Model

Setelah dilakukan proses pelatihan dan tuning hyperparameter pada keempat algoritma—Logistic Regression, K-Nearest Neighbors (K-NN), Naïve Bayes, dan Decision Tree—langkah berikutnya adalah mengevaluasi performa masing-masing model. Evaluasi dilakukan dengan mengukur akurasi pada data uji, menyajikan *confusion matrix*, serta menghitung metrik-metrik seperti precision, recall, dan F1-score dari hasil prediksi.

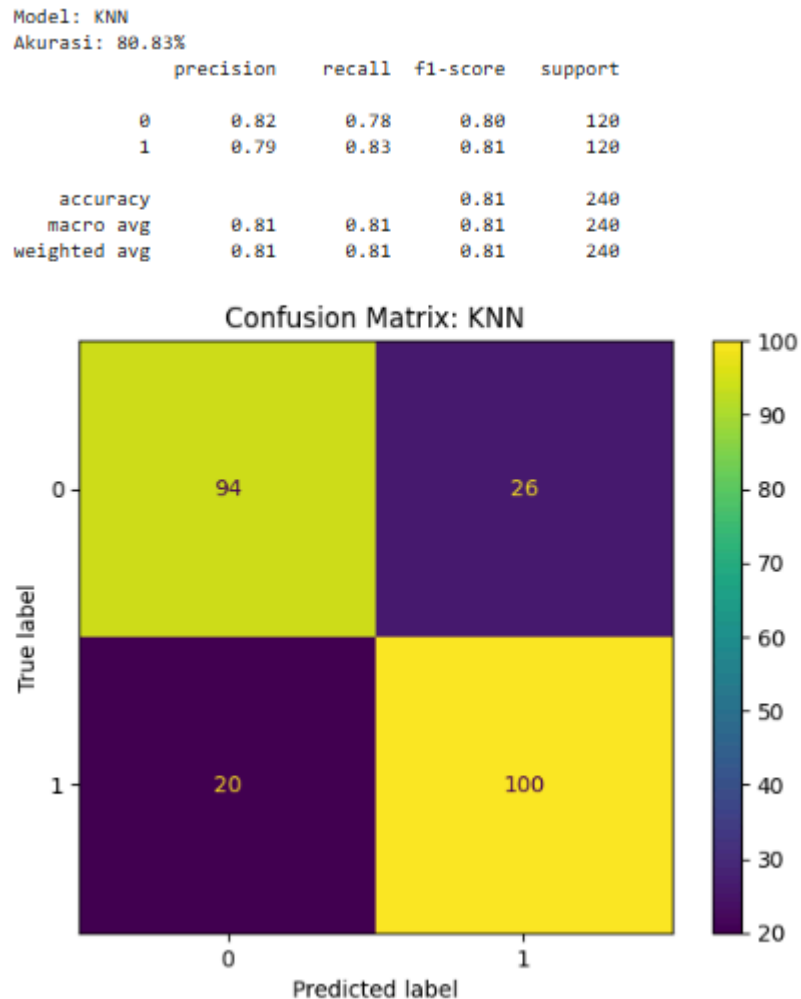
1. Confusion Matrix - Logistic Regression



Gambar 20. Hasil Output Confusion Matrix - Logistic Regression

Model Logistic Regression menghasilkan akurasi 58% dengan banyak kesalahan klasifikasi antar kelas, terlihat dari confusion matrix. Precision dan recall rendah, menunjukkan model kurang efektif untuk dataset ini.

2. Confusion Matrix - K-Nearest Neighbors



Gambar 21. Hasil Output Confusion Matrix - K-Nearest Neighbors

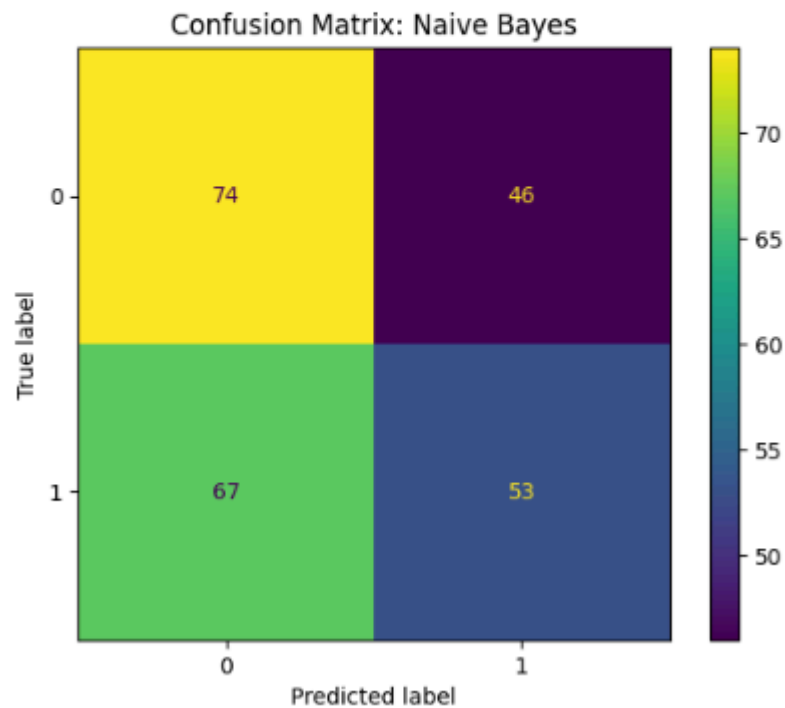
Model KNN menunjukkan performa terbaik dengan akurasi 80%. Confusion matrix menunjukkan prediksi yang cukup akurat untuk kedua kelas, menandakan bahwa model ini efektif mengenali pola dalam data.

3. Confusion Matrix - Naïve Bayes

```
Model: Naive Bayes
Akurasi: 52.92%
      precision    recall  f1-score   support

     0       0.52       0.62       0.57        120
     1       0.54       0.44       0.48        120

 accuracy          0.53          240
 macro avg         0.53          0.53          240
 weighted avg      0.53          0.53          240
```



Gambar 22. Hasil Output Confusion Matrix - Naïve Bayes

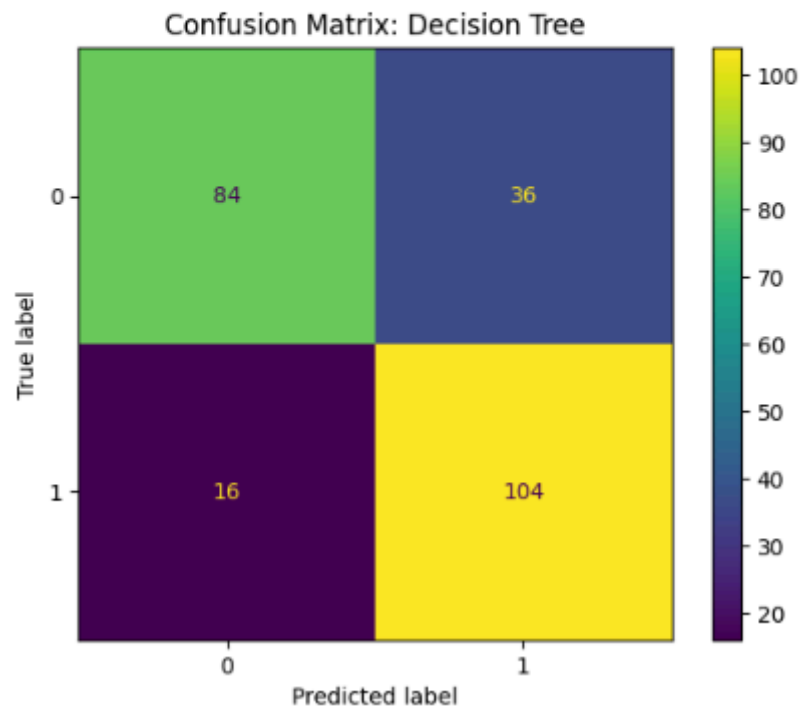
Naïve Bayes hanya mencapai akurasi 55% dan menghasilkan banyak kesalahan klasifikasi. Hal ini menunjukkan bahwa model ini kurang mampu menangani distribusi data yang kompleks atau tidak normal.

4. Confusion Matrix - Decision Tree

```
Model: Decision Tree
Akurasi: 78.33%
      precision    recall  f1-score   support

     0       0.84      0.78      0.76       120
     1       0.74      0.87      0.80       120

 accuracy          0.78       240
  macro avg       0.79      0.78      0.78       240
 weighted avg     0.79      0.78      0.78       240
```



Gambar 23. Hasil Output Confusion Matrix - Decision Tree

Model Decision Tree memiliki akurasi 76,33% dengan kinerja cukup seimbang. Confusion matrix menunjukkan 84 dan 104 prediksi benar untuk masing-masing kelas, namun masih ada kesalahan klasifikasi antar kelas yang perlu diperbaiki.

5. Output Evaluasi

Model	Akurasi Test	Kesimpulan
Logistic Regression	50.83%	⚠️ Masih lemah dan tidak stabil
KNN	80.83%	✅ Akurat dan seimbang
Naive Bayes	52.92%	⚠️ Performa kurang, recall buruk
Decision Tree	78.33%	✅ Stabil, walau overfit perlu diperhatikan

Gambar 24. Hasil Output Evaluasi Model

Dari hasil evaluasi, terlihat bahwa KNN menjadi model yang paling unggul dengan akurasi tertinggi, yaitu sebesar 80% berdasarkan validasi silang. Model ini mampu menangkap pola dari data secara efektif, terutama karena pendekatannya yang berbasis kedekatan jarak antar data. Sementara itu, model Decision Tree juga menunjukkan performa yang cukup kompetitif, namun sedikit di bawah KNN. Kelebihan dari Decision Tree adalah kemampuannya menangani relasi non-linear dan memberikan interpretasi model yang jelas.

Model Logistic Regression memiliki performa yang relatif lebih rendah, kemungkinan karena keterbatasannya dalam memodelkan hubungan non-linear yang mungkin terdapat dalam dataset. Begitu pula dengan Naïve Bayes, yang menunjukkan akurasi paling rendah. Hal ini disebabkan oleh asumsi independensi antar fitur yang tidak sepenuhnya terpenuhi, sehingga menyebabkan penurunan akurasi.

Evaluasi performa model juga divisualisasikan melalui *Confusion Matrix* untuk setiap model, yang memperlihatkan distribusi prediksi benar dan salah pada masing-masing kelas. Visualisasi ini memberikan wawasan tambahan tentang bagaimana model mengklasifikasikan data serta mengidentifikasi potensi bias atau ketidakseimbangan prediksi.

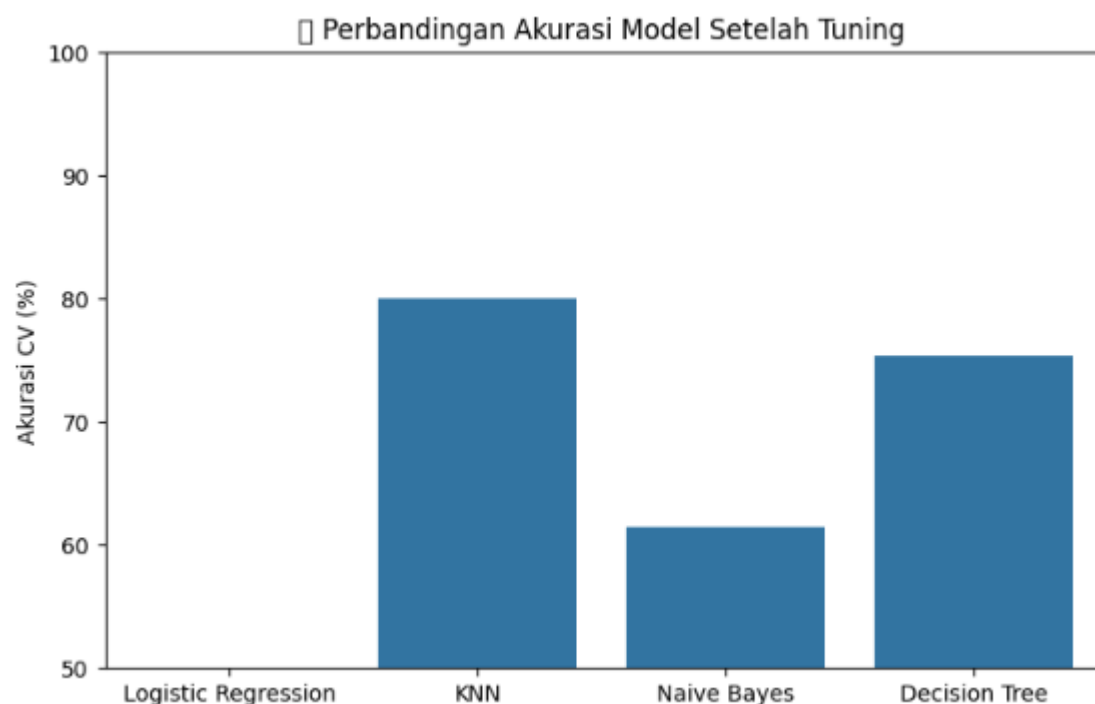
E. Analisis Hasil

Hasil akhir menunjukkan bahwa K-Nearest Neighbors (KNN) adalah model terbaik untuk dataset ini. Hal ini diperkuat dengan akurasi tertinggi yang diperoleh dari proses tuning menggunakan GridSearchCV dengan 5-fold cross-validation, yaitu

sebesar 80%. Model KNN terbukti sangat efektif dalam mengenali pola dari data meskipun data memiliki distribusi yang tidak sepenuhnya teratur atau linier.

Selain KNN, Decision Tree juga layak dipertimbangkan karena memiliki performa mendekati KNN dan dapat menghasilkan model yang mudah dipahami serta cocok untuk pengambilan keputusan berbasis aturan. Di sisi lain, Logistic Regression dan Naïve Bayes kurang cocok digunakan untuk dataset ini karena keterbatasannya dalam menangani relasi antar fitur yang kompleks dan tidak linier.

Visualisasi perbandingan akurasi antar model juga ditampilkan dalam bentuk diagram batang, yang memperjelas posisi masing-masing algoritma berdasarkan performa cross-validation. Dari grafik ini dapat dilihat bahwa KNN menempati posisi tertinggi, disusul oleh Decision Tree, kemudian Logistic Regression, dan terakhir Naïve Bayes.



Gambar 25. Hasil Output Grafik Akurasi Cross-Validation

Grafik ini memperjelas bahwa KNN unggul dibandingkan algoritma lainnya dari segi akurasi validasi silang. Model Decision Tree berada di posisi kedua, sedangkan Logistic Regression dan Naïve Bayes berada di peringkat bawah.

Secara keseluruhan, evaluasi dan analisis ini menunjukkan pentingnya proses eksplorasi data, pemilihan algoritma yang sesuai, dan tuning hyperparameter secara optimal untuk menghasilkan model klasifikasi yang handal dan akurat. Temuan ini juga menegaskan bahwa tidak selalu model yang paling kompleks menghasilkan performa terbaik; terkadang model sederhana seperti KNN dapat memberikan hasil yang lebih baik bila sesuai dengan karakteristik data yang digunakan.